RozRides — Complete Booking System (Cash-Based)

THE MENTAL MODEL FIRST

Before writing a single line of code, understand what a booking actually is in the RozRides context.

A booking is a **confirmed agreement** between two strangers mediated by your platform. One person is trusting another with a physical asset worth hundreds of thousands of rupees. Your booking system is not just a calendar feature — it is the **entire trust and coordination framework** of your business.

The cash-based model is actually simpler and more appropriate for Pakistan's market. Here is how money flows:

text

RENTER ARRIVES AT PICKUP

↓

Renter pays Security Deposit CASH to Host

↓

Host hands over car keys

↓

Trip happens

↓

Renter returns car

↓

Post-trip inspection done together

↓

No damage → Host returns full deposit CASH to Renter

Damage found → Host deducts repair cost from deposit

↓

Renter pays Rental Amount CASH to Host

↓

Trip closed on app

Your app's role is not to handle money. Your app's role is to:

- Coordinate and confirm the booking
- Document the car condition before and after
- Provide a paper trail for both parties
- Give admin visibility into all transactions
- Resolve disputes using documented evidence

Every design decision flows from this understanding.

THE COMPLETE BOOKING LIFECYCLE — BIRD'S EYE VIEW

text

```
DISCOVERY       INTENT        NEGOTIATION      COMMITMENT
_________       ______        ____________     ___________

Renter browses  →  Selects dates  →  Submits request →  Host accepts
car listings      checks price      sees car rules    booking confirmed
                  sees cash info    optional message  dates blocked


EXECUTION       VERIFICATION     CLOSURE        AFTERMATH
_________       ____________     _______        _________

Both meet at   →  Pre-trip      →  Car returned   →  Reviews posted
pickup location   inspection done   post inspection   host rates renter
cash deposit      keys handed       cash rent paid    renter rates car
paid to host      trip goes active  deposit returned
```

Every single step in that chain needs a screen, a database state, and a notification. Let's build each one.

PART 1 — PRE-BOOKING: DISCOVERY TO INTENT

1.1 — The Car Detail Screen (Modified)

This is where the booking journey begins. The renter is looking at a car and thinking "I want this."

What currently exists:
The car detail screen shows specs and photos.

What needs to be added:
Everything below the specs section becomes a booking entry point with clear cash payment instructions.

The screen layout from top to bottom:

text

```
┌──────────────────────────────┐
│ [Car Photos Carousel]        │
│                              │
│ Toyota Corolla 2020          │
│ ⭐ 4.8  (24 reviews)         │
│ DHA Phase 5, Lahore          │
│                              │
│ ──────── Car Specs ──────────  │
│ Automatic | Petrol | 5 Seats   │
│ AC | Bluetooth | GPS         │
│                              │
│ ──────── Pricing ────────────  │
```

```
│ PKR 3,500 / day          │
│ Security Deposit: PKR 10,000  │
│                          │
│ 💵 All payments are CASH      │
│   paid directly to host      │
│                          │
│ ──────── Availability ──────── │
│ [CALENDAR WIDGET HERE]       │
│                          │
│ ──────── Trip Estimate ──────── │
│ 3 days × PKR 3,500 = 10,500  │
│ Security deposit  = 10,000  │
│ ──────────────────────────── │
│ Bring to pickup   = 20,500  │
│ (Cash — paid to host)        │
│                          │
│ ──────── How Cash Works ──────── │
│ 1. Pay deposit at pickup     │
│ 2. Take the car              │
│ 3. Return car at end of trip  │
│ 4. Pay rent, get deposit back  │
│                          │
│ ──────── Host Info ──────────── │
│ [Host Photo] Ahmed K.        │
│ ✓ Verified  ⭐ 4.9 host rating │
│ Responds within 2 hours      │
```

```
│                          │
│ ──────── Car Rules ──────────── │
│ • No smoking               │
│ • No pets                  │
│ • Return with full tank      │
│                          │
│ ──────── Reviews ─────────────── │
│ [Review cards]             │
│                          │
│ [   REQUEST TO BOOK   ]    │
└──────────────────────────────────┘
```

The "Request to Book" button behavior:

The button is DISABLED and shows "Select Dates" until the renter has picked valid start and end dates on the calendar. Once dates are selected and confirmed available, the button activates.

1.2 — The Availability Calendar Widget

This is the most technically important piece of the pre-booking UI. Get this right and everything downstream becomes easier.

Visual design:

text

```
      January 2025

 Mo  Tu  We  Th  Fr  Sa  Su

         1   2   3   4   5

  6   7   8   9  10  11  12

 13  14  [15][16][17] 18  19   ← [15-17] = already booked (grayed)

 20  21  22  23  24  25  26
```

27 28 29 30 31

▶ Tap start date, then end date

■ Gray = Unavailable

■ Green = Your selection

How the calendar gets its data:

When the car detail screen loads, you make ONE Firestore query to get all confirmed booked date ranges for this car:

dart

```
// booking_service.dart


Future<List<DateRange>> getBookedDatesForCar(String carId) async {


  // Only confirmed and active bookings block dates

  // Pending requests do NOT block dates yet

  // This is intentional — pending requests are just expressions

  // of interest, not confirmed holds


  final snapshot = await FirebaseFirestore.instance

    .collection('bookings')

    .where('carId', isEqualTo: carId)

    .where('status', whereIn: ['confirmed', 'active'])

    .get();


  return snapshot.docs.map((doc) {

    return DateRange(
```

```
    start: (doc['startDate'] as Timestamp).toDate(),
    end: (doc['endDate'] as Timestamp).toDate(),
  );
 }).toList();
}
```

The overlap check — the most critical function in your entire app:

dart

```
bool isDatesAvailable({
  required DateTime requestedStart,
  required DateTime requestedEnd,
  required List<DateRange> existingBookings,
}) {
  for (final booking in existingBookings) {

    // Interval overlap formula:
    // Two ranges overlap if start1 < end2 AND end1 > start2

    bool overlaps = requestedStart.isBefore(booking.end) &&
                    requestedEnd.isAfter(booking.start);

    if (overlaps) return false; // Conflict — dates taken
  }
  return true; // No conflicts — dates are free
}
```

Why this formula works — explained simply:

Imagine booking A is Jan 15–17. A new request for Jan 16–18 overlaps:

- New start (16) is before existing end (17) → TRUE
- New end (18) is after existing start (15) → TRUE
- Both true = overlap = conflict

What about Jan 18–20 after a Jan 15–17 booking?

- New start (18) is before existing end (17) → FALSE
- One condition false = no overlap = available

The double-booking race condition:

Two renters see the same car. Both see Jan 15–17 as available. Both press "Request to Book" at the same time. Without protection, two pending requests exist for the same dates. This is fine — pending requests don't block dates. But when the HOST confirms one, the system must immediately invalidate the other.

This is handled by a Firestore Transaction at the moment of host confirmation. More on this in Part 3.

1.3 — The Cash Pricing Calculator

No payment processing. Just clear math displayed to the user.

dart

```
// lib/utils/pricing_calculator.dart


class CashPricingBreakdown {
  final int totalDays;
  final double pricePerDay;
  final double totalRent;        // pricePerDay × totalDays
  final double securityDeposit;   // Fixed, set by host
  final double bringToPickup;     // securityDeposit only (pay deposit first)
  final double payOnReturn;       // totalRent (pay rent when returning)
  final double depositRefund;     // securityDeposit (returned if no damage)
```

```
  const CashPricingBreakdown({
    required this.totalDays,
    required this.pricePerDay,
    required this.totalRent,
    required this.securityDeposit,
    required this.bringToPickup,
    required this.payOnReturn,
    required this.depositRefund,
  });
}

class PricingCalculator {
  static CashPricingBreakdown calculate({
    required DateTime startDate,
    required DateTime endDate,
    required double pricePerDay,
    required double securityDeposit,
  }) {
    final totalDays = endDate.difference(startDate).inDays;

    if (totalDays <= 0) throw Exception('Invalid date range');

    final totalRent = pricePerDay * totalDays;

    return CashPricingBreakdown(
```

```
    totalDays: totalDays,
    pricePerDay: pricePerDay,
    totalRent: totalRent,
    securityDeposit: securityDeposit,
    bringToPickup: securityDeposit,   // Only deposit at pickup
    payOnReturn: totalRent,          // Only rent at return
    depositRefund: securityDeposit,
  );
 }
}
```

Display this breakdown in two clear sections:

text

```
┌──────────────────────────────┐
│ Cash Payment Schedule        │
│                              │
│ 💵 AT PICKUP (you pay host)  │
│ Security Deposit   = PKR 10,000│
│                              │
│ 💵 AT RETURN (you pay host)  │
│ PKR 3,500 × 3 days  = PKR 10,500│
│                              │
│ 💵 AT RETURN (host pays you) │
│ Deposit refund     = PKR 10,000│
│ (if no damage)               │
│                              │
│ ────────────────────────────│
```

```
│ Net cost to you    = PKR 10,500│
│                    │
│ ℹ All cash, paid directly     │
│   to and from the host        │
└──────────────────────────────┘
```

PART 2 — THE BOOKING REQUEST CREATION

2.1 — Booking Summary Screen

Before submitting, the renter sees a final confirmation screen. This is their last chance to review everything. No payment details — just trip details and the cash schedule.

Screen: lib/screens/booking/booking_summary_screen.dart

text

```
┌──────────────────────────────┐
│ ← Confirm Your Booking       │
│                              │
│ [Car thumbnail]              │
│ Toyota Corolla 2020          │
│ DHA Phase 5, Lahore          │
│ Host: Ahmed Karimi           │
│                              │
│ ──────── Trip Dates ──────── │
│ Pickup:   Mon, Jan 15, 2025  │
│ Return:   Thu, Jan 17, 2025  │
│ Duration: 2 days             │
│                              │
```

```
│ ──────── Your Profile ──────── │
│ [Your photo] Ali Hassan        │
│ ✓ CNIC Verified                │
│                                │
│ ──────── Cash Schedule ──────── │
│                                │
│ 📍 AT PICKUP                   │
│ Pay host: PKR 10,000 (deposit) │
│                                │
│ 📍 AT RETURN                   │
│ Pay host: PKR 7,000 (2 days)   │
│ Receive:  PKR 10,000 (deposit) │
│        if no damage found      │
│                                │
│ ──────── Car Rules ──────────── │
│ • No smoking                   │
│ • Return with full tank        │
│                                │
│ ──────── Cancellation ──────── │
│ Free cancellation until        │
│ Jan 14, 2025 at 11:59 PM       │
│                                │
│ By tapping confirm, you agree  │
│ to RozRides Terms of Service.  │
│ All payments are cash between  │
│ you and the host directly.     │
```

```
│                          │
│  [   SEND BOOKING REQUEST  ]  │
└──────────────────────────────┘
```

2.2 — The Booking Creation Function

dart

```dart
// lib/services/booking_service.dart


Future<String> createBookingRequest({
  required String carId,
  required String hostId,
  required DateTime startDate,
  required DateTime endDate,
  required CashPricingBreakdown pricing,
  String? messageToHost,
}) async {
  final renterId = FirebaseAuth.instance.currentUser!.uid;


  // Step 1: Verify renter is CNIC verified
  final renterDoc = await FirebaseFirestore.instance
      .collection('users').doc(renterId).get();


  if (renterDoc.data()!['cnicStatus'] != 'approved') {
    throw Exception('CNIC_NOT_VERIFIED');
    // Show dialog: "Complete identity verification to book cars"
  }
```

```
// Step 2: Final availability check
final isAvailable = await checkAvailability(carId, startDate, endDate);
if (!isAvailable) {
  throw Exception('DATES_UNAVAILABLE');
}

// Step 3: Create the booking document
final bookingRef = FirebaseFirestore.instance
    .collection('bookings').doc();

final bookingData = {
  'bookingId': bookingRef.id,
  'carId': carId,
  'hostId': hostId,
  'renterId': renterId,

  // Dates
  'startDate': Timestamp.fromDate(startDate),
  'endDate': Timestamp.fromDate(endDate),
  'totalDays': pricing.totalDays,

  // Cash pricing — stored for record keeping and dispute reference
  'pricePerDay': pricing.pricePerDay,
  'totalRent': pricing.totalRent,
  'securityDeposit': pricing.securityDeposit,
```

```
// Cash payment tracking — host confirms receipt manually
'cashPayments': {
  'depositPaidToHost': false,      // Host confirms at pickup
  'depositPaidAmount': 0,
  'depositPaidAt': null,
  'rentPaidToHost': false,         // Host confirms at return
  'rentPaidAmount': 0,
  'rentPaidAt': null,
  'depositRefundedToRenter': false, // Host confirms refund
  'depositRefundedAmount': 0,
  'depositRefundedAt': null,
},

// Status
'status': 'pending',
'createdAt': FieldValue.serverTimestamp(),
'updatedAt': FieldValue.serverTimestamp(),
'expiresAt': Timestamp.fromDate(
    DateTime.now().add(const Duration(hours: 24))
),

// Optional renter message to host
'messageToHost': messageToHost ?? '',

// Review tracking
```

```
  'reviewStatus': {
    'renterSubmitted': false,
    'hostSubmitted': false,
  },

  // Cancellation policy copied from car listing
  'cancellationPolicy': 'flexible',
};

await bookingRef.set(bookingData);

// Step 4: Create a chat conversation for this booking
await createConversation(bookingRef.id, hostId, renterId, carId);

// Step 5: Log the initial timeline event
await logBookingEvent(
  bookingRef.id, 'pending', renterId, 'Booking request created'
);

// Step 6: Notify host
await sendNotificationToUser(
  userId: hostId,
  title: '🚗 New Booking Request!',
  body: 'Someone wants to rent your car. Respond within 24 hours.',
  data: {'bookingId': bookingRef.id, 'screen': 'request_detail'},
);
```

```
  return bookingRef.id;
}
```

After this function completes, navigate to a confirmation screen:

text

```
┌──────────────────────────────┐
│                              │
│      ✓ Request Sent!         │
│                              │
│ Your booking request has been │
│ sent to Ahmed. He has 24 hours │
│ to respond.                  │
│                              │
│ We'll notify you the moment  │
│ he accepts or declines.      │
│                              │
│ While you wait, remember to  │
│ have PKR 10,000 cash ready   │
│ for the security deposit.    │
│                              │
│ Expires: Tue, Jan 14 at 3:45PM │
│                              │
│ [  VIEW MY BOOKINGS  ]       │
│ [  BACK TO HOME     ]        │
│                              │
└──────────────────────────────┘
```

PART 3 — THE HOST RESPONSE FLOW

3.1 — Host Notification

The moment a booking request is created, the host receives:

Push notification:

text

🚗 New Booking Request!

Ali Hassan wants to rent your Corolla

Jan 15–17 • PKR 10,500 rental income

Tap to respond →

In-app notification (existing notifications_screen.dart, extended):
Same content with a deep link directly to the request detail screen.

3.2 — Host Incoming Requests Screen

Screen: lib/screens/host/incoming_requests_screen.dart

text

```
┌──────────────────────────────┐
│ Booking Requests             │
│ ○ Pending (2)  ○ All         │
│                              │
│ ┌──────────────────────────┐ │
│ │ [Ali photo] Ali Hassan   │ │
│ │ ✓ CNIC Verified          │ │
│ │ 🗓️ Jan 15–17 (2 days)    │ │
│ │ 🚗 Toyota Corolla        │ │
```

```
│ │ 💵 You receive: PKR 10,500│ │
│ │   cash at return       │ │
│ │ ⏱ Expires in 18 hours   │ │
│ │                        │ │
│ │[DECLINE]   [ACCEPT]    │ │
│ └──────────────────────────┘ │
│                              │
│ ┌──────────────────────────┐ │
│ │[Sara photo] Sara Malik   │ │
│ │ ✓ CNIC Verified         │ │
│ │ 📅 Jan 20–22 (2 days)    │ │
│ │ 🚗 Toyota Corolla       │ │
│ │ 💵 You receive: PKR 10,500│ │
│ │   cash at return       │ │
│ │ ⏱ Expires in 22 hours   │ │
│ │                        │ │
│ │[DECLINE]   [ACCEPT]    │ │
│ └──────────────────────────┘ │
└──────────────────────────────┘
```

Tapping a request card opens the full request detail screen:

text

```
┌──────────────────────────────┐
│ ← Request from Ali Hassan    │
│                              │
│ ──────── Renter Profile ─────  │
```

```
│ [Large photo]            │
│ Ali Hassan               │
│ ✓ CNIC Verified          │
│ Member since: Dec 2024   │
│ Previous trips: 3        │
│ Rating as renter: ⭐ 4.7 │
│ [View Full Profile]      │
│                          │
│ ──────── Booking Details ──── │
│ Car: Toyota Corolla 2020 │
│ Pickup:  Wed, Jan 15 at 10 AM │
│ Return:  Fri, Jan 17 at 10 AM │
│ Duration: 2 days         │
│                          │
│ ──────── Cash You Receive ──── │
│ At pickup (deposit):     │
│   PKR 10,000 cash from renter │
│                          │
│ At return (rent):        │
│   PKR 7,000 cash from renter │
│                          │
│ At return (refund if no dmg): │
│   PKR 10,000 cash back to │
│   renter                 │
│                          │
│ Your net income: PKR 7,000 │
```

```
│                          │
│ ──────── Renter's Message ──  │
│ "Hi, I'll pick up in the      │
│ morning. Is 10 AM okay?"      │
│                          │
│ [   DECLINE REQUEST   ]     │
│ [   ACCEPT REQUEST    ]     │
└──────────────────────────────┘
```

3.3 — Host Accept Flow

When the host taps "Accept":

Confirmation dialog:

text

```
┌──────────────────────────────┐
│ Accept this booking?         │
│                          │
│ You're confirming Ali Hassan  │
│ for Jan 15–17.               │
│                          │
│ Remember:                    │
│ • Collect PKR 10,000 deposit  │
│   when Ali arrives           │
│ • Collect PKR 7,000 rental    │
│   when Ali returns the car    │
│ • Return deposit if no damage │
│                          │
```

```
│  [Cancel]   [Yes, Accept]     │
└──────────────────────────────────┘
```

The accept function — runs as a Cloud Function:

JavaScript

```
// functions/index.js


exports.acceptBooking = functions.https.onCall(async (data, context) => {
  const { bookingId } = data;
  const hostId = context.auth.uid;


  await admin.firestore().runTransaction(async (transaction) => {


    const bookingRef = admin.firestore()
      .collection('bookings').doc(bookingId);
    const bookingSnap = await transaction.get(bookingRef);
    const booking = bookingSnap.data();


    // Verify caller is the host
    if (booking.hostId !== hostId) {
      throw new Error('UNAUTHORIZED');
    }


    // Verify booking is still pending
    if (booking.status !== 'pending') {
      throw new Error('BOOKING_NO_LONGER_PENDING');
    }
```

```
const carRef = admin.firestore()
   .collection('cars').doc(booking.carId);
const carSnap = await transaction.get(carRef);

// Check availability one final time inside the transaction
// This prevents race conditions with other simultaneous accepts
const currentBookedRanges = carSnap.data().bookedDateRanges || [];
const requestedStart = booking.startDate.toDate();
const requestedEnd = booking.endDate.toDate();

for (const range of currentBookedRanges) {
  const rangeStart = range.start.toDate();
  const rangeEnd = range.end.toDate();
  const overlaps = requestedStart < rangeEnd && requestedEnd > rangeStart;

  if (overlaps) {
    throw new Error('DATES_NOW_UNAVAILABLE');
    // Rolls back entire transaction
  }
}

// No conflicts — confirm the booking
transaction.update(bookingRef, {
  'status': 'confirmed',
  'updatedAt': admin.firestore.FieldValue.serverTimestamp(),
```

```
  });

  // Block the dates on the car document
  transaction.update(carRef, {
    'bookedDateRanges': admin.firestore.FieldValue.arrayUnion([{
      'start': booking.startDate,
      'end': booking.endDate,
      'bookingId': bookingId,
    }])
  });

  // Log timeline event
  const timelineRef = bookingRef.collection('timeline').doc();
  transaction.set(timelineRef, {
    'status': 'confirmed',
    'triggeredBy': hostId,
    'timestamp': admin.firestore.FieldValue.serverTimestamp(),
    'note': 'Host accepted the booking request',
  });
});

// Outside transaction: auto-decline competing requests
await autoDeclineOverlappingRequests(bookingId);

// Notify renter
await sendNotification(booking.renterId, {
```

```
    title: '✓ Booking Confirmed!',
    body: `Ahmed accepted your booking for Jan 15–17.
        Bring PKR 10,000 cash for the deposit.` ,
    data: { bookingId, screen: 'booking_detail' }
  });

  return { success: true };
});
```

Auto-declining overlapping requests:

JavaScript

```
async function autoDeclineOverlappingRequests(confirmedBookingId) {
  const confirmedBooking = await admin.firestore()
    .collection('bookings').doc(confirmedBookingId).get();

  const { carId, startDate, endDate } = confirmedBooking.data();

  // Find all other pending bookings for this car
  const pendingBookings = await admin.firestore()
    .collection('bookings')
    .where('carId', '==', carId)
    .where('status', '==', 'pending')
    .where('bookingId', '!=', confirmedBookingId)
    .get();

  const batch = admin.firestore().batch();
```

```
for (const doc of pendingBookings.docs) {
  const pending = doc.data();
  const pendingStart = pending.startDate.toDate();
  const pendingEnd = pending.endDate.toDate();
  const confirmedStart = startDate.toDate();
  const confirmedEnd = endDate.toDate();

  const overlaps = pendingStart < confirmedEnd &&
                 pendingEnd > confirmedStart;

  if (overlaps) {
    batch.update(doc.ref, {
      'status': 'rejected',
      'rejectionReason': 'HOST_CHOSE_ANOTHER_BOOKING',
      'updatedAt': admin.firestore.FieldValue.serverTimestamp(),
    });

    // Notify these renters their request was declined
    await sendNotification(pending.renterId, {
      title: 'Booking Not Available',
      body: 'Sorry, the host accepted another booking for these dates.',
      data: { bookingId: doc.id }
    });
  }
}
```

```
  await batch.commit();
}
```

3.4 — Host Decline Flow

When the host taps "Decline", show a reason selector:

text

```
┌──────────────────────────────┐
│ Why are you declining?       │
│                              │
│ ○ Car unavailable (personal  │
│   use on these dates)        │
│                              │
│ ○ Car needs maintenance      │
│                              │
│ ○ Renter profile incomplete  │
│                              │
│ ○ I found a better booking   │
│                              │
│ ○ Other                      │
│   [Type reason here...]      │
│                              │
│ [Cancel]   [Decline Request] │
└──────────────────────────────┘
```

3.5 — The 24-Hour Auto-Expiry

Hosts who don't respond kill the renter experience. A scheduled Cloud Function handles this:

JavaScript

```
exports.expireStaleBookings = functions.pubsub
  .schedule('every 30 minutes')
  .onRun(async (context) => {

    const now = admin.firestore.Timestamp.now();

    const staleBookings = await admin.firestore()
      .collection('bookings')
      .where('status', '==', 'pending')
      .where('expiresAt', '<=', now)
      .get();

    const batch = admin.firestore().batch();

    for (const doc of staleBookings.docs) {
      batch.update(doc.ref, {
        'status': 'expired',
        'updatedAt': now,
      });

      // Notify renter
      await sendNotification(doc.data().renterId, {
        title: 'Request Expired',
```

```
    body: 'The host did not respond in time. Browse similar cars.',
    data: { bookingId: doc.id }
  });
}

  await batch.commit();
});
```

Send host a warning at the 20-hour mark:

text

⏱️ Booking Request Expiring Soon!

You have 4 hours left to respond to Ali's request.

After that it auto-expires and he can book elsewhere.

PART 4 — THE CONFIRMED BOOKING STATE

Once confirmed, both parties can see the full booking details and communicate via chat.

4.1 — Renter's My Bookings Screen

Screen: lib/screens/renter/my_bookings_screen.dart

text

```
┌──────────────────────────────┐
│ My Bookings                  │
│                              │
│ ─── UPCOMING ──────────────  │
│ ┌────────────────────────┐   │
│ │[Car img] Toyota Corolla │  │
│ │ ✅ CONFIRMED           │  │
```

```
│ │ Jan 15–17 • 2 days      │ │
│ │ Host: Ahmed K.          │ │
│ │ 💵 Bring PKR 10,000 cash │ │
│ │   for deposit at pickup │ │
│ │ [Message Host] [Details] │ │
│ └──────────────────────────┘ │
│                              │
│ ─── PENDING ──────────────── │
│ ┌──────────────────────────┐ │
│ │ [Car img] Honda City     │ │
│ │ ⏱ AWAITING HOST (12h)    │ │
│ │ Jan 20–22 • 2 days       │ │
│ │ [Cancel Request]         │ │
│ └──────────────────────────┘ │
│                              │
│ ─── PAST ─────────────────── │
│ ┌──────────────────────────┐ │
│ │ [Car img] Suzuki Alto    │ │
│ │ ✓ COMPLETED              │ │
│ │ Dec 10–12                │ │
│ │ ⭐ Leave a Review        │ │
│ └──────────────────────────┘ │
└──────────────────────────────┘
```

4.2 — Booking Detail Screen

Screen: lib/screens/renter/booking_detail_screen.dart

text

```
┌──────────────────────────────┐
│ ← Booking #RR-2025-0042      │
│                              │
│ STATUS: ✅ CONFIRMED         │
│                              │
│ ──────── Car Details ──────── │
│ [Car photo]                  │
│ Toyota Corolla 2020          │
│ DHA Phase 5, Lahore          │
│                              │
│ ──────── Trip Dates ──────── │
│ Pickup:  Wed, Jan 15 at 10 AM │
│ Return:  Fri, Jan 17 at 10 AM │
│ Duration: 2 days             │
│                              │
│ ──────── Host Contact ──────── │
│ [Ahmed photo] Ahmed Karimi   │
│ ⭐ 4.9 host rating           │
│ [💬 Message Ahmed]           │
│                              │
│ ──────── Cash Schedule ──────── │
│                              │
│ 📍 AT PICKUP — Jan 15        │
│ ○ Pay host PKR 10,000 deposit │
│   (have cash ready!)         │
```

```
│                          │
│ 📍 AT RETURN — Jan 17      │
│ ○ Pay host PKR 7,000 rental  │
│ ○ Receive PKR 10,000 deposit  │
│   back (if no damage)      │
│                          │
│ ──────── Cancellation ────────  │
│ Free cancellation until     │
│ Jan 14 at 11:59 PM         │
│ [Cancel Booking]           │
│                          │
│ ──────── Booking Timeline ──  │
│ ✓ Request sent    Jan 12 2PM │
│ ✓ Host accepted   Jan 12 4PM │
│ ○ Trip starts     Jan 15    │
│ ○ Trip ends       Jan 17    │
└──────────────────────────────┘
```

PART 5 — THE HANDOVER: CONFIRMED → ACTIVE

The handover is when the car physically changes hands and cash changes hands for the first time. This is the most critical real-world moment.

5.1 — Day-of-Pickup Reminder

On the morning of the pickup date:

Renter gets:

text

🚗 Your trip starts today!

Toyota Corolla — Pickup at 10 AM

📍 DHA Phase 5, Lahore

💵 Bring PKR 10,000 cash for deposit

[View Booking Details]

Host gets:

text

🚗 Ali Hassan arrives today!

Toyota Corolla — Pickup at 10 AM

💵 Collect PKR 10,000 deposit at handover

[Start Handover Checklist]

5.2 — Pre-Trip Inspection Flow

Who initiates it: The host, in the presence of the renter.

The host's booking detail screen shows a prominent banner on pickup day:

text

```
┌──────────────────────────────┐
│ 🚗 Pickup Day — Ali Hassan   │
│                              │
│ Before handing over the keys: │
│ 1. Collect PKR 10,000 deposit │
│ 2. Complete inspection together│
│ 3. Both parties sign         │
│                              │
│ [START HANDOVER CHECKLIST]   │
```

└──────────────────────────────────────┘

The inspection flow:
lib/screens/trip/pre_trip_inspection_screen.dart

text

Step 1 of 7: Confirm Deposit Received

"Has Ali Hassan paid the security deposit?"

Amount received: [PKR _____] (host enters amount)

○ Yes, I received PKR 10,000 cash

○ Renter will pay later (not recommended)

[NEXT →]

text

Step 2 of 7: Exterior Front

"Take a photo of the car's front"

[📷 Camera button]

[Photo added ✓]

Any existing damage to note?

○ No damage

○ Yes → [Describe damage...]

[NEXT →]

text

Step 3 of 7: Exterior Rear

[Same structure]

Step 4 of 7: Exterior Left Side

[Same structure]

Step 5 of 7: Exterior Right Side

[Same structure]

Step 6 of 7: Interior

[Same structure]

Step 7 of 7: Fuel & Odometer

Current fuel level:

○ Full ○ 3/4 ○ 1/2 ○ 1/4 ○ Empty

Odometer reading: [_____] km

[NEXT →]

Final step — both parties sign:

text

```
┌──────────────────────────────┐
│ Handover Confirmation        │
│                              │
```

```
│ Summary:                  │
│ ✓ Deposit collected: PKR 10,000│
│ ✓ 6 photos taken          │
│ ✓ No existing damage noted    │
│ ✓ Fuel: Full              │
│ ✓ Odometer: 45,231 km       │
│                           │
│ Both parties confirm the above │
│ is accurate before keys are   │
│ handed over.              │
│                           │
│ Host (Ahmed):             │
│ [Tap here to sign] ✓        │
│                           │
│ Renter (Ali):             │
│ [Tap here to sign] ✓        │
│                           │
│ [COMPLETE HANDOVER — START TRIP]│
└──────────────────────────────────┘
```

When "Complete Handover" is tapped:

dart

```
Future<void> startTrip(String bookingId, PreTripInspection inspection) async {
  await FirebaseFirestore.instance.runTransaction((transaction) async {


    final bookingRef = FirebaseFirestore.instance
        .collection('bookings').doc(bookingId);
```

```
  // Update booking status to active
  transaction.update(bookingRef, {
    'status': 'active',
    'tripStartedAt': FieldValue.serverTimestamp(),
    'updatedAt': FieldValue.serverTimestamp(),
    // Record cash deposit confirmation
    'cashPayments.depositPaidToHost': true,
    'cashPayments.depositPaidAmount': inspection.depositCollected,
    'cashPayments.depositPaidAt': FieldValue.serverTimestamp(),
  });

  // Store pre-trip inspection
  final inspectionRef = bookingRef.collection('inspections').doc('pre_trip');
  transaction.set(inspectionRef, inspection.toMap());

  // Log timeline event
  final timelineRef = bookingRef.collection('timeline').doc();
  transaction.set(timelineRef, {
    'status': 'active',
    'triggeredBy': FirebaseAuth.instance.currentUser!.uid,
    'timestamp': FieldValue.serverTimestamp(),
    'note': 'Car handed over. Deposit collected. Trip started.',
  });
});
```

```
  // Post system message in chat
  await postSystemMessage(
    bookingId: bookingId,
    message: '🚗 Trip started — Car handed over on Jan 15 at 10:23 AM. '
             'Deposit of PKR 10,000 confirmed received. Drive safe!',
  );
}
```

PART 6 — THE ACTIVE TRIP

6.1 — Active Trip Screen for Renter

Screen: lib/screens/trip/active_trip_screen.dart

text

```
┌──────────────────────────────────┐
│ Your Active Trip                 │
│                                  │
│ [Car photo — full width]         │
│                                  │
│ Toyota Corolla 2020              │
│ Registration: LEA-1234           │
│                                  │
│ ┌──────────────────────────┐     │
│ │ RETURN IN                │     │
│ │ 1 day  14 hrs  32 min    │     │
│ └──────────────────────────┘     │
│                                  │
```

```
│ Return by: Fri Jan 17, 10 AM  │
│ Return to: DHA Phase 5, Lahore │
│                               │
│ ──────── At Return ──────────  │
│ 💵 Pay host: PKR 7,000 cash   │
│ 💵 Receive:  PKR 10,000 back  │
│   (your deposit, if no dmg)   │
│                               │
│ ──────── Quick Actions ──────  │
│ [💬 Message Host]             │
│ [📞 Emergency: 0300-ROZRIDE]  │
│                               │
│ ──────── Trip Info ──────────  │
│ Started:  Jan 15 at 10:23 AM  │
│ Fuel at pickup: Full          │
│ Odometer at pickup: 45,231 km │
│                               │
│ ──────── Car Rules ──────────  │
│ • Return with full tank       │
│ • No smoking in vehicle       │
└───────────────────────────────┘
```

Active Trip Screen for Host:

text

```
┌───────────────────────────────┐
│ Car Currently Rented          │
│                               │
```

```
│ [Car photo]              │
│ Toyota Corolla 2020      │
│                          │
│ Rented to: Ali Hassan ✓  │
│ Deposit collected: PKR 10,000 │
│                          │
│ ┌──────────────────────┐ │
│ │ RETURNS IN           │ │
│ │ 1 day  14 hrs  32 min │ │
│ └──────────────────────┘ │
│                          │
│ Return: Fri Jan 17, 10 AM │
│                          │
│ ──────── At Return ──────── │
│ 💵 Collect: PKR 7,000 rent │
│ 💵 Refund:  PKR 10,000 deposit │
│   (if no damage found)   │
│                          │
│ [💬 Message Ali]         │
└──────────────────────────┘
```

Late Return Escalation:

text

0 hours overdue → Renter: "Your trip has ended. Return car now."

→ Host: "Ali's trip ended. Car not yet returned."

1 hour overdue → Renter: "Car is 1 hour overdue. Contact host."

→ Host: "Car 1hr overdue. Additional charges may apply."

3 hours overdue → Admin is automatically notified

→ Both parties: "RozRides support has been alerted."

PART 7 — THE RETURN: ACTIVE → COMPLETED

This is where cash changes hands for the second time. The app must guide both parties through this carefully.

7.1 — Return Day Reminders

Morning of return day:

Renter gets:

text

🏁 Return Day!

Toyota Corolla due back by 10 AM

📍 DHA Phase 5, Lahore

💵 Bring PKR 7,000 cash for rental

💵 You'll receive PKR 10,000 back

(your deposit, if no damage)

[View Booking]

Host gets:

text

🏁 Car Returns Today!

Ali Hassan returns Corolla by 10 AM

💵 Collect PKR 7,000 rental cash

💵 Refund PKR 10,000 deposit

(after inspection — if no damage)

[Start Return Inspection]

7.2 — Post-Trip Inspection Flow

Who initiates: Either party when the car is back.

The post-trip inspection is identical in structure to the pre-trip inspection with one critical difference: the app automatically compares each item against the pre-trip record and highlights discrepancies.

Step 1 — Fuel check with comparison:

text

```
┌──────────────────────────────┐
│ Step 1 of 6: Fuel Level      │
│                              │
│ At pickup it was: FULL ←──── │
│ Current level: [selector]    │
│ ○ Full ○ 3/4 ○ 1/2           │
│ ○ 1/4  ○ Empty               │
│                              │
│ ⚠ Car must be returned with  │
│   full tank per car rules    │
└──────────────────────────────┘
```

Step 2–6 — Damage comparison view:

text

```
┌──────────────────────────────┐
│ Step 2 of 6: Exterior Front  │
│                              │
│ BEFORE (at pickup):          │
│ [Pre-trip photo]             │
│ ✓ No damage noted            │
│                              │
│ AFTER (now):                 │
│ [📷 Take new photo]          │
│ [Photo taken ✓]              │
│                              │
│ Any NEW damage to report?    │
│ ○ No new damage              │
│ ○ Yes → [Describe damage...] │
│                              │
└──────────────────────────────┘
```

The comparison runs automatically in the app:

dart

```
InspectionResult compareInspections({
  required Map<String, dynamic> preTrip,
  required Map<String, dynamic> postTrip,
}) {
  final List<String> issues = [];
  bool hasNewDamage = false;
```

```
// Check fuel level
if (preTrip['fuelLevel'] == 'full' &&
   postTrip['fuelLevel'] != 'full') {
  issues.add('Fuel not returned full — '
          'was: Full, now: ${postTrip["fuelLevel"]}');
}

// Check odometer for extreme overuse
final preOdo = preTrip['odometerReading'] as int;
final postOdo = postTrip['odometerReading'] as int;
final kmDriven = postOdo - preOdo;

// Check each area for new damage
final areas = ['front', 'rear', 'leftSide', 'rightSide', 'interior'];

for (final area in areas) {
  final preDamage = preTrip['items'][area]['hasDamage'] as bool;
  final postDamage = postTrip['items'][area]['hasDamage'] as bool;

  if (!preDamage && postDamage) {
    hasNewDamage = true;
    issues.add(
      'New damage on $area: '
      '${postTrip["items"][area]["notes"]}'
    );
  }
```

```
  }

  return InspectionResult(
    issues: issues,
    hasNewDamage: hasNewDamage,
    kmDriven: kmDriven,
  );
}
```

7.3 — The Cash Settlement Step

After the inspection photos are taken but before both parties sign, the app walks them through the cash settlement:

NO DAMAGE SCENARIO:

text

```
┌──────────────────────────────┐
│ ✓ Inspection Complete        │
│ No new damage found!         │
│                              │
│ ──────── Cash Settlement ──  │
│                              │
│ Step 1: Renter pays host     │
│ PKR 7,000 rental (2 days)    │
│ [Host confirms received] ☐   │
│                              │
│ Step 2: Host refunds renter  │
│ PKR 10,000 security deposit  │
```

```
│ [Renter confirms received] ☐   │
│                                │
│ Both boxes must be checked     │
│ before you can close the trip  │
│                                │
└────────────────────────────────┘
```

DAMAGE FOUND SCENARIO:

text

```
┌────────────────────────────────┐
│ ⚠ New Damage Found             │
│                                │
│ • Scratch on rear bumper       │
│                                │
│ ──────── Cash Settlement ────  │
│                                │
│ Step 1: Renter pays host       │
│ PKR 7,000 rental (2 days)      │
│ [Host confirms received] ☐     │
│                                │
│ Step 2: Deposit Deduction      │
│ Damage amount (agree on):      │
│ PKR [_______] (host enters)    │
│                                │
│ Deposit: PKR 10,000            │
│ Deduction: PKR [_______]       │
│ ────────────────────────────── │
```

```
│ Renter gets back: PKR [auto]   │
│                                │
│ [Renter agrees to deduction] ☐ │
│ [Host refunds remaining] ☐     │
│                                │
│ If you cannot agree:           │
│ [RAISE A DISPUTE WITH ROZRIDES]│
└────────────────────────────────┘
```

The dispute button creates a damageClaims document and alerts admin with full inspection evidence — both photo sets, the comparison, and the disagreement details.

7.4 — Final Sign-Off and Trip Closure

After cash is settled:

text

```
┌────────────────────────────────┐
│ Close This Trip                │
│                                │
│ Both parties confirm:          │
│                                │
│ ✓ Car returned in agreed       │
│   condition                    │
│ ✓ Rental cash settled          │
│ ✓ Deposit settled              │
│                                │
│ Host (Ahmed):                  │
│ [Tap to confirm & sign] ✓      │
```

```
│                          │
│  Renter (Ali):            │
│  [Tap to confirm & sign] ✓     │
│                          │
│  [  CLOSE TRIP  ]          │
└──────────────────────────────┘
```

The trip completion function:

dart

```
Future<void> completeTrip({
  required String bookingId,
  required PostTripInspection inspection,
  required CashSettlement settlement,
}) async {
  await FirebaseFirestore.instance.runTransaction((transaction) async {

    final bookingRef = FirebaseFirestore.instance
        .collection('bookings').doc(bookingId);

    // Get the car document to unblock dates
    final bookingSnap = await transaction.get(bookingRef);
    final booking = bookingSnap.data()!;

    final carRef = FirebaseFirestore.instance
        .collection('cars').doc(booking['carId']);

    // Update booking to completed
```

```
transaction.update(bookingRef, {
  'status': 'completed',
  'tripEndedAt': FieldValue.serverTimestamp(),
  'updatedAt': FieldValue.serverTimestamp(),

  // Record cash settlement
  'cashPayments.rentPaidToHost': true,
  'cashPayments.rentPaidAmount': settlement.rentPaid,
  'cashPayments.rentPaidAt': FieldValue.serverTimestamp(),
  'cashPayments.depositRefundedToRenter': true,
  'cashPayments.depositRefundedAmount': settlement.depositRefunded,
  'cashPayments.depositRefundedAt': FieldValue.serverTimestamp(),
  'cashPayments.damageDeduction': settlement.damageDeduction,
});

// Store post-trip inspection
final inspectionRef = bookingRef
    .collection('inspections').doc('post_trip');
transaction.set(inspectionRef, inspection.toMap());

// Unblock the dates on the car
transaction.update(carRef, {
  'bookedDateRanges': FieldValue.arrayRemove([{
    'start': booking['startDate'],
    'end': booking['endDate'],
    'bookingId': bookingId,
```

```
    }])
  });

  // Log timeline
  final timelineRef = bookingRef.collection('timeline').doc();
  transaction.set(timelineRef, {
    'status': 'completed',
    'triggeredBy': FirebaseAuth.instance.currentUser!.uid,
    'timestamp': FieldValue.serverTimestamp(),
    'note': 'Trip completed. Cash settled. Car returned.',
  });
});

 // Post system message in chat
 await postSystemMessage(
  bookingId: bookingId,
  message: '✅ Trip completed on Jan 17 at 10:45 AM. '
         'All cash settled. Thanks for using RozRides!',
 );

 // Schedule review prompts — send after 2 hours
 await scheduleReviewPrompts(bookingId);
}
```

PART 8 — CANCELLATIONS

Cancellations are cash-based too. Since no money has been processed through the app, cancellation is simpler — it is just a status update with a notification.

8.1 — Cancellation Rules

text

WHO CANCELS	WHEN	RESULT
Renter	Before host accepts	Booking cancelled. No money involved. No penalty.
Renter	After confirmed, >24h before pickup	Free cancellation. No money has changed hands yet.
Renter	After confirmed, <24h before pickup	Renter forfeits their right to cancel free. Host keeps goodwill — no app-enforced penalty on cash system. HOWEVER: note this on the renter's profile as a "late cancellation" strike.
Host	Any time before pickup	Host cancels. Renter notified. Host receives a platform strike on their profile. 3 strikes = account review.

After pickup Trip has started Cannot cancel. Must go

(status = active) through early return flow.

Cancellation strike system on host profile:

dart

```
// stored on user document
'hostStats': {
  'totalBookings': 24,
  'cancellationsByHost': 2,      // Host-initiated cancels
  'cancellationStrikeCount': 2,   // Strikes towards review
  'lastCancellationAt': Timestamp,
}
```

Show this on the host's public profile: "2 host cancellations" — transparency builds trust for renters.

8.2 — Cancellation Screen

Screen: lib/screens/booking/cancellation_screen.dart

text

```
┌──────────────────────────────┐
│ ← Cancel Booking             │
│                              │
│ Toyota Corolla 2020          │
│ Jan 15–17                    │
│                              │
│ ──────── Cancellation Info ── │
│                              │
│ Since no payment was processed │
```

```
│ through the app, there is no   │
│ automatic refund needed.       │
│                                │
│ If you paid any cash already,  │
│ contact the host directly to   │
│ arrange its return.            │
│                                │
│ ⚠ This action cannot be undone.│
│                                │
│ Why are you cancelling?        │
│ ○ Change of plans              │
│ ○ Found a better option        │
│ ○ Emergency                    │
│ ○ Host was unresponsive        │
│ ○ Other: [text field]          │
│                                │
│ [  CONFIRM CANCELLATION  ]     │
└────────────────────────────────┘
```

PART 9 — THE DISPUTE SYSTEM

Without a payment gateway to hold funds, disputes are about evidence and reputation. Your dispute system must capture enough information to make a fair judgment and update user reputation accordingly.

9.1 — When Disputes Are Raised

A dispute can be raised in two situations:

Situation A — Damage disagreement at return:
Host claims damage that renter denies. They cannot agree on a deduction amount. One party taps "Raise a Dispute."

Situation B — Deposit not refunded:
Renter claims host refused to return the deposit without valid reason.

9.2 — Dispute Data Model

text

```
damageClaims/{claimId}
  ├── bookingId: String
  ├── carId: String
  ├── claimedBy: String (userId of who raised dispute)
  ├── claimType: String
  │   "damage_dispute" | "deposit_not_refunded" | "other"
  ├── description: String
  ├── hostClaimedAmount: Number (what host wants to deduct)
  ├── renterAgreedAmount: Number (what renter agrees to)
  ├── status: String
  │   "open" | "admin_reviewing" | "resolved_for_host"
  │   "resolved_for_renter" | "resolved_mutually"
  ├── adminNotes: String (nullable)
  ├── resolvedAt: Timestamp (nullable)
  ├── resolvedBy: String (adminId, nullable)
  └── createdAt: Timestamp
```

Evidence is automatically attached from the booking:

- Pre-trip inspection photos

- Post-trip inspection photos
- Full chat conversation log
- Booking timeline

Admin reviews all of this in the /dashboard/claims panel and makes a ruling. The ruling is posted to both parties via notification and in-app message.

9.3 — Dispute Resolution Outcomes

text

RESOLVED FOR HOST:

→ Admin confirms damage is real and fair

→ Renter's profile gets a "damage incident" note

→ Host's reputation unaffected

RESOLVED FOR RENTER:

→ Admin finds host claim unjustified

→ Host's profile gets a "disputed deposit" note

→ If host has 2+ such marks: account flagged for review

RESOLVED MUTUALLY:

→ Admin suggests a middle-ground amount

→ Both parties confirm agreement in app

→ No reputation impact

PART 10 — COMPLETE DATA MODEL

text

bookings/{bookingId}

```
├── bookingId: String
├── carId: String
├── hostId: String
├── renterId: String
│
├── startDate: Timestamp
├── endDate: Timestamp
├── totalDays: Integer
│
├── pricePerDay: Number
├── totalRent: Number
├── securityDeposit: Number
│
├── cashPayments: Map
│   ├── depositPaidToHost: Boolean
│   ├── depositPaidAmount: Number
│   ├── depositPaidAt: Timestamp (nullable)
│   ├── rentPaidToHost: Boolean
│   ├── rentPaidAmount: Number
│   ├── rentPaidAt: Timestamp (nullable)
│   ├── depositRefundedToRenter: Boolean
│   ├── depositRefundedAmount: Number
│   ├── depositRefundedAt: Timestamp (nullable)
│   └── damageDeduction: Number
│
```

```
├── status: String
│   "pending" | "confirmed" | "active"
│   "completed" | "cancelled" | "rejected"
│   "expired" | "disputed"
│
├── messageToHost: String
├── cancellationPolicy: String
├── cancellationReason: String (nullable)
├── rejectionReason: String (nullable)
│
├── tripStartedAt: Timestamp (nullable)
├── tripEndedAt: Timestamp (nullable)
│
├── reviewStatus: Map
│   ├── renterSubmitted: Boolean
│   └── hostSubmitted: Boolean
│
├── createdAt: Timestamp
├── updatedAt: Timestamp
└── expiresAt: Timestamp
```

bookings/{bookingId}/inspections/pre_trip

```
├── depositCollected: Number
├── fuelLevel: String
├── odometerReading: Number
```

```
  ├── hostSigned: Boolean
  ├── renterSigned: Boolean
  ├── completedAt: Timestamp
  └── items: Map
      ├── front: { hasDamage: Boolean, notes: String, photos: [URL] }
      ├── rear: { hasDamage: Boolean, notes: String, photos: [URL] }
      ├── leftSide: { hasDamage: Boolean, notes: String, photos: [URL] }
      ├── rightSide: { hasDamage: Boolean, notes: String, photos: [URL] }
      └── interior: { hasDamage: Boolean, notes: String, photos: [URL] }

bookings/{bookingId}/inspections/post_trip
  └── (same structure + damageDeduction field)

bookings/{bookingId}/timeline/{eventId}
  ├── status: String
  ├── triggeredBy: String (userId or "system")
  ├── timestamp: Timestamp
  └── note: String
```

PART 11 — STATUS BADGE SYSTEM

Consistent visual language across the entire app:

dart

```
// lib/utils/booking_status_utils.dart

Color getStatusColor(String status) {
```

```
  switch (status) {
    case 'pending':   return Colors.orange;
    case 'confirmed': return Colors.green;
    case 'active':    return Colors.blue;
    case 'completed': return Colors.grey;
    case 'cancelled': return Colors.red;
    case 'rejected':  return Colors.red;
    case 'expired':   return Colors.red;
    case 'disputed':  return Colors.purple;
    default:          return Colors.grey;
  }
}

String getStatusLabel(String status) {
  switch (status) {
    case 'pending':   return '⏱️ Awaiting Host';
    case 'confirmed': return '✅ Confirmed';
    case 'active':    return '🚗 Trip Active';
    case 'completed': return '✓ Completed';
    case 'cancelled': return '✗ Cancelled';
    case 'rejected':  return '✗ Declined by Host';
    case 'expired':   return '✗ Request Expired';
    case 'disputed':  return '⚠️ Under Review';
    default:          return 'Unknown';
  }
```

```
}

String getStatusDescription(String status) {
  switch (status) {
    case 'pending':
      return 'Waiting for host to accept or decline';
    case 'confirmed':
      return 'Booking accepted — prepare cash for pickup';
    case 'active':
      return 'Trip in progress — deposit collected';
    case 'completed':
      return 'Trip finished — all cash settled';
    case 'cancelled':
      return 'Booking was cancelled';
    case 'rejected':
      return 'Host declined this request';
    case 'expired':
      return 'Host did not respond in time';
    case 'disputed':
      return 'RozRides support is reviewing this trip';
    default:
      return '';
  }
}
```

PART 12 — COMPLETE FILE STRUCTURE

text

```
lib/
 ├── services/
 │   └── booking_service.dart
 │       createBookingRequest()
 │       acceptBooking()       ← calls Cloud Function
 │       declineBooking()      ← calls Cloud Function
 │       cancelBooking()
 │       startTrip()
 │       completeTrip()
 │       raiseDispute()
 │       getBookedDates()
 │       checkAvailability()
 │
 ├── utils/
 │   ├── pricing_calculator.dart
 │   │     CashPricingBreakdown
 │   │     PricingCalculator.calculate()
 │   └── booking_status_utils.dart
 │       getStatusColor()
 │       getStatusLabel()
 │       getStatusDescription()
 │
 ├── models/
 │   ├── booking_model.dart
 │   ├── pre_trip_inspection_model.dart
```

```
│   ├── post_trip_inspection_model.dart
│   ├── cash_settlement_model.dart
│   └── damage_claim_model.dart
│
├── widgets/
│   ├── availability_calendar.dart
│   ├── booking_status_badge.dart
│   ├── cash_schedule_widget.dart     ← shows pickup/return cash amounts
│   ├── booking_card.dart
│   └── inspection_comparison_widget.dart
│
└── screens/
    ├── booking/
    │   ├── booking_summary_screen.dart
    │   ├── booking_confirmed_screen.dart
    │   └── cancellation_screen.dart
    │
    ├── host/
    │   ├── incoming_requests_screen.dart
    │   ├── request_detail_screen.dart
    │   └── host_calendar_screen.dart
    │
    ├── renter/
    │   ├── my_bookings_screen.dart
    │   └── booking_detail_screen.dart
```

```
│
└── trip/
    ├── active_trip_screen.dart
    ├── pre_trip_inspection_screen.dart
    ├── post_trip_inspection_screen.dart
    ├── cash_settlement_screen.dart    ← NEW — guides cash exchange
    └── dispute_screen.dart            ← NEW — raises dispute with admin
```

THE COMPLETE STATE MACHINE — CASH VERSION

text

```
USER ACTION              APP WRITES              CASH HAPPENS
____________             ___________             _____________

Renter selects dates   →  [READ] bookedDateRanges      Nothing
Renter submits request →  [CREATE] booking:pending     Nothing
Host accepts           →  [UPDATE] booking:confirmed   Nothing
                          [UPDATE] car:bookedRanges
Host declines          →  [UPDATE] booking:rejected    Nothing
24h timeout            →  [UPDATE] booking:expired     Nothing

Both meet at pickup    →  [UPDATE] booking:active      Renter pays
Pre-inspection done       [CREATE] inspections/pre     PKR 10,000
Both sign                 cashPayments.depositPaid     deposit CASH
                                                       to host

Trip ends, car returned →  [CREATE] inspections/post   Renter pays
```

Post-inspection done compareInspections() PKR X,XXX
rent CASH
to host

No damage found → [UPDATE] booking:completed Host refunds
Both sign off cashPayments.rentPaid full deposit
cashPayments.depositRefunded CASH to renter

Damage found, agreed → [UPDATE] booking:completed Host deducts
Both sign off damageDeduction recorded agreed amount,
refunds rest

Damage found, disputed → [CREATE] damageClaim:open Cash on hold
Admin reviews Admin sees all evidence until resolved
Admin rules → [UPDATE] damageClaim:resolved Winner gets
reputation updated their cash